# Homework 1

for (CSED490V) Foundations of Continuous Computation

Jungwoo Kim
Department of Mathematics
Pohang University of Science and Technology

When you prove the computability of a real number or a real function, you should explicitly provide an algorithm that is required in the definition of computability. In the algorithm, you can only use rational numbers, basic operations over rationals (i.e., $+$, $-$, $\times$, $/$, abs), comparisons between rationals (e.g., $=$, $<$, $\leq$), and basic programming constructs (i.e., `if`, `for`, `while`).

**Problem 1.** In this problem, "computable" means Borel-computable.

(a) Prove that if $x, y \in \mathbb{R}$ are computable, then $x + y$ is computable.

(b) Prove that $\sqrt{k}$ is computable for all integers $k \geq 2$.
*[Hint]* Define $(x_m)_{m\in\mathbb{N}_0} \subseteq \mathbb{Q}$ by $x_0 \coloneqq k$ and

$$x_{m+1} \coloneqq \frac{1}{2}\left(x_m + \frac{k}{x_m}\right).$$

Show that there exists $\alpha > 0$ such that $0 < x_m - \sqrt{k} < k\alpha^m$ for all $m \in \mathbb{N}_0$. Use this result to prove the original claim.

*Solution.*

(a) Let $p, r\colon \mathbb{N}_0 \to \mathbb{Q}$ be computable sequences satisfying

$$|p(n) - x| < 2^{-n}, \qquad |r(n) - y| < 2^{-n} \quad (n \in \mathbb{N}_0).$$

On input $n \in \mathbb{N}_0$, compute $p(n+1)$ and $r(n+1)$ and return $q(n) \coloneqq p(n+1) + r(n+1)$. This algorithm uses only rational addition and calls to the given computable approximation procedures. Moreover,

$$|q(n) - (x+y)| \leq |p(n+1) - x| + |r(n+1) - y| < 2^{-(n+1)} + 2^{-(n+1)} = 2^{-n}.$$

Hence $(q(n))_{n\in\mathbb{N}_0}$ computes $x + y$.

(b) Fix an integer $k \geq 2$, and define $x_0 = k$ and

$$x_{m+1} = \frac{1}{2}\left(x_m + \frac{k}{x_m}\right) \qquad (m \in \mathbb{N}_0).$$

The algorithm starts at the rational number $k$ and computes each successive $x_m$ by positive rational arithmetic. Put $s = \sqrt{k}$. Because $k > s > 0$,

$$x_{m+1} - s = \frac{x_m^2 - 2sx_m + s^2}{2x_m} = \frac{(x_m - s)^2}{2x_m} > 0$$

whenever $x_m > s$. By induction, $x_m > s$ for every $m$. Since $0 < x_m - s < x_m$, this identity also gives

$$0 < x_{m+1} - s < \frac{x_m - s}{2}.$$

Iteration yields

$$0 < x_m - s \leq \frac{k-s}{2^m} < \frac{k}{2^m} \qquad (m \in \mathbb{N}_0),$$

where the strict inequality at $m = 0$ follows from $k - s < k$. Thus the requested estimate holds with $\alpha = \frac{1}{2}$.

Given $n \in \mathbb{N}_0$, find an integer $m \geq 0$ by starting at $m = 0$ and incrementing $m$ while $k2^{-m} \geq 2^{-n}$. This search terminates because repeated division by 2 sends $k2^{-m}$ to 0, and its test uses only rational comparisons. Return the rational number $x_m$, computed by $m$ Newton steps. Then

$$|x_m - \sqrt{k}| < k2^{-m} < 2^{-n},$$

proving the Borel computability of $\sqrt{k}$.

□

**Problem 2.** Prove that the following problem is decidable (i.e., there is an algorithm solving the problem): Given computable functions $f, g\colon \mathbb{N} \to \mathbb{Q}$ representing distinct computable reals $x, y \in \mathbb{R}$ (i.e., $|f(n) - x| < 2^{-n}$ and $|g(n) - y| < 2^{-n}$ for all $n \in \mathbb{N}$, and $x \neq y$), decide whether $x < y$ or $x > y$.

*Solution.* For $n = 1, 2, 3, \ldots$, compute $a \coloneqq f(n)$, $b \coloneqq g(n)$, and $r \coloneqq 2^{-n}$, and perform the following tests:

(1) If $a + r \leq b - r$, return $x < y$.

(2) If $b + r \leq a - r$, return $x > y$.

(3) Otherwise, increment $n$ and repeat.

Only rational arithmetic, comparisons, and a loop are required. If the first condition holds, the approximation guarantees imply

$$x < a + r \leq b - r < y,$$

and hence the answer is correct; the second condition is symmetric.

To prove termination, let $d \coloneqq |x - y| > 0$. Choose $n$ sufficiently large that $4 \cdot 2^{-n} < d$. If $x < y$, then

$$f(n) + 2^{-n} < x + 2 \cdot 2^{-n} < y - 2 \cdot 2^{-n} < g(n) - 2^{-n},$$

so the first test succeeds. If $x > y$, the symmetric argument makes the second test succeed. Thus the algorithm always halts and returns the correct order under the promise $x \neq y$.

□

**Problem 3.**

(a) Prove that $\exp\colon \mathbb{R} \to \mathbb{R}$ is computable over the entire $\mathbb{R}$. Your proof should be self-contained: it must include all details, and must not refer to any part of the proof covered in class.

(b) See `p3.ipynb`.

*Solution.*

(a) We give a rational-only oracle algorithm, including an explicit computable error budget, that works for every $x \in \mathbb{R}$. Let $\alpha\colon \mathbb{N}_0 \to \mathbb{Q}$ be an arbitrary rational name of $x$, so that $|\alpha(i) - x| < 2^{-i}$ for all $i \in \mathbb{N}_0$. Fix an output precision $n \in \mathbb{N}_0$.

First query $q_0 \coloneqq \alpha(0)$. Starting at $B = 3$ and increasing $B$ by 1 as needed, find an integer $B$ such that

$$B > |q_0| + 2.$$

The search uses rational comparisons and terminates. For every $i \in \mathbb{N}_0$, the approximation inequalities give

$$|x| < |q_0| + 1 < B, \qquad |\alpha(i)| < |x| + 1 < B.$$

Set $C \coloneqq 3^B \in \mathbb{N}$, computing it by $B$ successive multiplications by 3. To justify this bound without using a computability result for exp, observe that for $j \geq 2$, $j! \geq 2^{j-1}$, with a strict inequality for $j \geq 3$. Therefore

$$e = \sum_{j=0}^{\infty} \frac{1}{j!} < 2 + \sum_{j=2}^{\infty} 2^{-(j-1)} = 3.$$

Since exp is increasing and $\exp(B) = e^B$ for integer $B$,

$$0 < \exp(t) \leq \exp(B) = e^B < C \qquad (-B \leq t \leq B).$$

In particular, the mean value theorem shows that, for $u, v \in [-B, B]$,

$$|\exp(u) - \exp(v)| \leq C|u - v|. \tag{1}$$

Taylor's theorem with the Lagrange remainder, applied to $P_d(t) \coloneqq \sum_{j=0}^{d} t^j/j!$, similarly gives

$$|\exp(t) - P_d(t)| \leq C\frac{B^{d+1}}{(d+1)!} \qquad (|t| < B). \tag{2}$$

Let $\eta \coloneqq 2^{-(n+2)} \in \mathbb{Q}$. Using a loop over nonnegative integers, find $m$ such that

$$C2^{-m} < \eta. \tag{3}$$

Next find $d \geq 0$ such that

$$C\frac{B^{d+1}}{(d+1)!} < \eta. \tag{4}$$

Both comparisons involve rationals. The first search terminates because $2^{-m} \to 0$; the second terminates because for $k \geq 2B$ the ratio of consecutive quantities $B^{k+1}/(k+1)!$ and $B^k/k!$ is $B/(k+1) < \frac{1}{2}$. One can maintain the rational quantity $CB^k/k!$ inductively, multiplying it by $B/(k+1)$ on each iteration; there is no need to evaluate a real exponential during the search.
Finally query $q \coloneqq \alpha(m)$ and return

$$y \coloneqq P_d(q) = \sum_{j=0}^{d} \frac{q^j}{j!} \in \mathbb{Q}.$$

This finite sum uses rational multiplication, division, and addition. By (1)–(4),

$$\begin{aligned} |y - \exp(x)| &\leq |P_d(q) - \exp(q)| + |\exp(q) - \exp(x)| \\ &< \eta + C2^{-m} < 2\eta = 2^{-(n+1)} < 2^{-n}. \end{aligned}$$

All searches terminate, and every numerical operation in the algorithm is rational arithmetic or comparison. Because $x$ and its name $\alpha$ were arbitrary, this proves computability of $\exp\colon \mathbb{R} \to \mathbb{R}$.

(b) The accompanying `p3.ipynb` implements the preceding algorithm as `creal_exp(x)`. It first obtains the integer bound $B$ from `x(0)`; it then chooses $m$ using (3), chooses $d$ using (4), and evaluates $P_d(\alpha(m))$ using an exact rational recurrence for its terms. The returned `CReal` computes this approximation afresh at each requested precision $n$. Thus its correctness and termination follow from part (a), for every valid input approximation sequence.

□

**Problem 4.** In this problem, you should prove claims using only the definition of computable functions. Do not use any other results, including those covered in class and in Problem 5.

(a) Prove that $f\colon \mathbb{R} \to \mathbb{R}$ defined by $f(x) \coloneqq$ (if $x \geq 0$ then 1 else 0) is not computable.

(b) Prove that $g\colon [0, \frac{1}{2}] \to [0, 1]$ defined by $g(x) \coloneqq x + \frac{1}{3}$ is not computable under Turing's definition: A function $h\colon [0, \frac{1}{2}] \to [0, 1]$ is called *computable in Turing's sense* if there exists an algorithm $\mathcal{A}$ that takes $(b_i)_{i \in \mathbb{N}_0} \subseteq \{0, 1\}$ and $j \in \mathbb{N}_0$, and returns 0 or 1 such that

$$h((0.b_0 b_1 \ldots)_2) = (0.c_0 c_1 \ldots)_2, \qquad c_j \coloneqq \mathcal{A}((b_i)_{i \in \mathbb{N}_0}, j).$$

This result shows that Turing's definition of computable functions is not equivalent to, and much more restrictive than, Borel's definition of computable functions.

*Solution.*

(a) Suppose for contradiction that an oracle algorithm $\mathcal{A}$ computes $f$ in the sense of the definition of a computable function. Give it precision $n = 2$ and the name $\alpha(i) = 0$ for every $i \in \mathbb{N}_0$; this names $x = 0$. The computation halts and returns some $q \in \mathbb{Q}$ with

$$|q - f(0)| = |q - 1| < \frac{1}{4}, \qquad \text{so } q > \frac{3}{4}.$$

Only finitely many indices of $\alpha$ were queried. Let $N$ be at least all of them (take $N = 0$ if none were queried), and put $t \coloneqq 2^{-(N+2)} > 0$. Define a second rational name by

$$\beta(i) \coloneqq \begin{cases} 0, & i \leq N, \\ -t, & i > N. \end{cases}$$

This is a name of $-t$: for $i \leq N$, $|\beta(i) - (-t)| = t < 2^{-i}$, and for $i > N$ the error is zero. The algorithm receives exactly the same answers to all oracle queries made during its run on $\alpha$, so its deterministic execution on $\beta$ also halts with output $q$. Yet $f(-t) = 0$, which requires $|q| < \frac{1}{4}$, contradicting $q > \frac{3}{4}$. Hence $f$ is not computable.

(b) Suppose that an algorithm $\mathcal{A}$ computes $g$ in the stated binary-digit sense. Consider the real number $x_0 \coloneqq \frac{1}{6}$. Its binary expansion is unique, since $\frac{1}{6}$ is not dyadic, and

$$g(x_0) = \frac{1}{2}.$$

Run $\mathcal{A}$ on the binary expansion of $x_0$ with output index $j = 0$. It must halt after inspecting only finitely many input digits and output a bit $c_0 \in \{0, 1\}$. Let $N$ be at least all inspected indices (take $N = 0$ if none were inspected). Because $x_0$ is not a dyadic rational, it lies strictly inside the dyadic interval specified by its first $N + 1$ binary digits. Consequently there are numbers $x^- < x_0 < x^+$ in $[0, \frac{1}{2}]$ that share those digits with $x_0$. We may choose these inputs to be non-dyadic so that their binary expansions are unique.
On each of these inputs, $\mathcal{A}$ must still return $c_0$, since the finite set of queried input digits has not changed. However,

$$g(x^-) < \frac{1}{2} < g(x^+).$$

Every binary expansion of a number strictly below $\frac{1}{2}$ has first fractional digit 0, whereas every binary expansion of a number strictly above $\frac{1}{2}$ (and at most 1) has first fractional digit 1. Thus correctness on $x^-$ requires $c_0 = 0$, while correctness on $x^+$ requires $c_0 = 1$, a contradiction. This argument does not depend on which of the two binary expansions of $\frac{1}{2}$ the algorithm chooses at $x_0$. Therefore $g$ is not Turing-computable as a function in the stated sense.

□

**Problem 5.** Prove that if $f\colon \mathbb{R} \to \mathbb{R}$ is computable (under Borel's definition), then $f$ is continuous.

*[Hint]* To prove that $f$ is continuous, it suffices to prove the following: for all $a \in \mathbb{R}$ and $n \in \mathbb{N}$, there exists $m \in \mathbb{N}$ such that $|x - a| < 2^{-m}$ implies $|f(x) - f(a)| < 2^{-n}$ for all $x \in \mathbb{R}$.

*Solution.* Let $\mathcal{A}$ be an oracle algorithm computing $f$. Fix an arbitrary $a \in \mathbb{R}$ and $n \in \mathbb{N}$. For every $i \in \mathbb{N}_0$, choose a rational number $\alpha(i)$ with

$$|\alpha(i) - a| < 2^{-(i+2)}.$$

Such rationals exist by density of $\mathbb{Q}$ in $\mathbb{R}$, even if $a$ is not computable. In particular $\alpha$ is a valid rational name of $a$. Run $\mathcal{A}$ on this name at precision $n + 2$. By computability it halts with a rational result $q$ satisfying

$$|q - f(a)| < 2^{-(n+2)}.$$

Let $I \subseteq \mathbb{N}_0$ be the finite set of oracle indices queried in this run. For each $i \in I$, the quantity $2^{-i} - |\alpha(i) - a|$ is strictly positive. Consequently, there exists $\delta > 0$ such that

$$|x - a| < \delta \quad \Longrightarrow \quad |\alpha(i) - x| < 2^{-i} \quad (i \in I).$$

Explicitly, take $\delta \coloneqq \min_{i \in I}(2^{-i} - |\alpha(i) - a|)$ when $I \neq \varnothing$, and $\delta \coloneqq 1$ otherwise.

For any $x \in \mathbb{R}$ with $|x - a| < \delta$, construct a rational name $\beta$ of $x$ by setting $\beta(i) = \alpha(i)$ for $i \in I$ and choosing, for each $i \notin I$, any rational $\beta(i)$ within $2^{-i}$ of $x$. This construction need not be uniformly computable: the definition requires $\mathcal{A}$ to work for every valid name. The names $\alpha$ and $\beta$ give identical answers to every query made in the first computation, so $\mathcal{A}(\beta, n + 2) = q$. It follows that

$$\begin{aligned} |f(x) - f(a)| &\leq |f(x) - q| + |q - f(a)| \\ &< 2^{-(n+2)} + 2^{-(n+2)} = 2^{-(n+1)} < 2^{-n}. \end{aligned}$$

Choose $m \in \mathbb{N}$ with $2^{-m} < \delta$. Then $|x - a| < 2^{-m}$ implies the required estimate for every $x \in \mathbb{R}$. Since $a$ and $n$ were arbitrary, $f$ is continuous on $\mathbb{R}$.

□